

Evidence-Constrained Neuro-Symbolic Ontology Repair for Semantic Drift in Versioned Normative Documents

Anonymous Author(s)

Abstract

Versioned normative documents continually revise definitions, obligations, exceptions, scopes, and effective conditions, causing OWL ontologies built from earlier versions to contain assertions that are no longer supported by current evidence. We present ECR-REPAIR, an evidence-constrained modular neuro-symbolic ontology repair framework that uses a Semantic Intermediate Representation (Semantic IR) as the neural-to-symbolic semantic interface. Large language models interpret normative evidence under candidate-content isolation; deterministic recovery and constraint-aware ranking perform symbolic semantic decision making; candidate-level neural entailment verification is invoked only when ranking remains ambiguous; and OWL/RDF materialization, reasoning, graph-delta validation, and frozen repair/CQ regression checks provide formal safety validation. We evaluate a conditional, evidence-relative semantic-drift repair task in which a potential repair event has already been localized, a relevant evidence window is provided, and the finite candidate set contains the correct repair; upstream drift discovery, open-corpus retrieval, and open-ended candidate generation are outside the primary metric. On an RFC-based blind-large benchmark constructed after method freeze, ECR-REPAIR achieves 1315/1320 (99.62%) closure accuracy and 262/264 (99.24%) strict event success over 264 events and 1320 runs, with zero wrong selections. Additional human temporal-semantic auditing, formal-closure stress testing, candidate-pool enlargement, a v5 cross-backend replication, and a full online candidate-identifier permutation test characterize the semantic, formal-safety, and robustness boundaries of the system. The results suggest that evidence-constrained ontology repair depends not only on stronger language-model outputs, but on an explicit, auditable, abstention-capable neuro-symbolic interface with clear responsibility boundaries.

CCS Concepts

• **Computing methodologies** → **Knowledge representation and reasoning**; • **Information systems** → *Semantic web*.

Keywords

ontology evolution, semantic drift, neuro-symbolic ontology repair, Semantic IR, large language models, OWL, selective prediction

ACM Reference Format:

Anonymous Author(s). 2026. Evidence-Constrained Neuro-Symbolic Ontology Repair for Semantic Drift in Versioned Normative Documents. In *Proceedings of Anonymous Review Submission (Anonymous Review)*. ACM, New York, NY, USA, 21 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 Introduction

Ontologies are widely used to represent standards, regulatory rules, protocol constraints, and domain knowledge. When their knowledge source is a normative document, correctness depends

not only on logical consistency but also on whether ontology assertions remain supported by the current version of the normative evidence. In practice, normative documents evolve continuously: a new version may supersede an older one, an exception may override a general rule, a qualifier may change the scope of several statements, or an attribute value may become obsolete after revision. Such changes can create semantic drift between an ontology and its normative source.

A central difficulty is that an outdated ontology assertion can remain syntactically valid and reasoner-consistent after a new specification is released, so a purely symbolic consistency check may raise no alarm. Conversely, asking an LLM to modify the ontology directly makes it difficult to answer three questions: which evidence justified the change, why one repair candidate was preferred, and under what conditions the system should refuse to modify the ontology. We therefore frame the core challenge as one of **responsibility allocation and auditability**. Neural components should be responsible for interpreting natural-language evidence; symbolic components should constrain that interpretation into inspectable decisions; formal closure should verify the structural, logical, and regression safety of the patch; and the system should be allowed to abstain whenever any layer lacks sufficient support.

Semantic drift is therefore different from an ordinary syntax error. An ontology may still be loadable by a reasoner and may remain logically consistent, while an attribute value, rule condition, or scope relation no longer matches the current specification. Textual differencing alone is insufficient, and OWL consistency checking alone is insufficient. A repair system must interpret version relations, rule-exception structure, and cross-sentence scope, and then convert that interpretation into a formally verifiable ontology edit.

We use a layered operational definition of semantic drift. Let

$\text{Support}(a, D_t) \in \{\text{supported}, \text{contradicted}, \text{underspecified}\}$

denote the evidence status of assertion a with respect to normative evidence at time or version t . If complete historical evidence is available, strict version drift requires t_0, t_1 and a replacement assertion a' such that

$$t_0 < t_1, \quad a' \neq a,$$

$$\text{Support}(a, D_{t_0}) = \text{supported}, \quad \text{Support}(a, D_{t_1}) \neq \text{supported},$$

$$\text{Support}(a', D_{t_1}) = \text{supported}.$$

The v5 benchmark, however, does not require every event to preserve an independently verifiable $t_0 \rightarrow t_1$ historical evidence chain. The primary evaluation therefore targets the narrower notion of **evidence-relative semantic-drift repair**. Let D_{cur} denote the frozen current evidence. The source assertion a_0 and the correct repair a^* satisfy

$$\text{Support}(a_0, D_{\text{cur}}) \neq \text{supported},$$

$$\exists a^* \in C_E : \quad \text{Support}(a^*, D_{\text{cur}}) = \text{supported}.$$

Temporal-Version events often encode explicit version or effective-date changes; General Rule–Exception and Cross-Sentence Scope events may instead expose rule or scope mismatches under the current evidence. This evidence-relative condition asks whether the current specification still supports the source assertion and whether the candidate set contains a current-evidence-supported replacement. It is *not* equivalent to reconstructing a complete historical $t_0 \rightarrow t_1$ evolution chain. A later strict provenance audit yields $N = 0$ under unrelaxed lineage and support criteria, so we do not characterize the primary benchmark as a provenance-complete historical-drift benchmark.

Large language models provide a natural mechanism for interpreting normative language, but direct LLM use introduces additional risks. A model may return structurally valid but semantically incorrect JSON, prefer a candidate because its description is clearer, exhibit order or identifier bias, or produce a definite repair when the evidence is insufficient. Instead of reducing the problem to “ask an LLM to choose the answer,” we decompose it into evidence interpretation, semantic representation, candidate mapping, selective gating, and formal validation.

ECR-REPAIR models repair as a modular neuro-symbolic process rather than a single LLM decision. The neural semantic module produces traceable interpretations of normative evidence. The symbolic semantic module constrains those interpretations into typed Semantic IR, deterministic temporal relations, and inspectable candidate scores. Candidate-level LLM entailment verification is reintroduced only when the symbolic ranking is ambiguous. Finally, OWL/RDF patching, reasoning, graph differencing, and frozen repair/CQ regression checks provide formal symbolic validation. Semantic IR therefore functions as the **neural-to-symbolic semantic interface**.

At the same time, ECR-REPAIR enforces **candidate-content isolation**: candidate values, operations, identifiers, scores, and top- k identities are not allowed to feed back into the semantic extraction or recovery modules. Ranking failure can return at most a content-free `ranking_failed=true` flag, which may trigger one additional recovery pass. This separation assigns “interpret the evidence” to neural components and “constrain and verify the decision” to symbolic components, rather than treating free-form LLM generation as an ontology update.

The research question is therefore:

How can a neuro-symbolic repair framework transform natural-language evidence from versioned normative documents into reliable OWL/RDF repair decisions while remaining evidence-constrained, candidate-content isolated, abstention-capable, and formally verifiable?

The main contributions are:

- We formulate ontology repair under evolving normative documents as an evidence-constrained finite-candidate selection problem followed by formal OWL closure validation.
- We propose ECR-REPAIR, a candidate-content-isolated modular neuro-symbolic architecture in which explicit Semantic IR connects LLM-based evidence interpretation, symbolic semantic constraints, candidate decision making, and OWL/RDF validation.

- We design a diagnosable pipeline of neural semantic interpretation $\rightarrow$ symbolic semantic decision $\rightarrow$ optional neural entailment verification $\rightarrow$ formal symbolic closure, with auditable recovery mechanisms for General Rule–Exception, Cross-Sentence Scope, and Temporal-Version ambiguity.
- After method freeze, we construct and freeze a larger RFC-based blind-large benchmark containing 264 events across 11 domains, 13 included RFC source families, and three semantic event types.
- We evaluate the framework with the main blind experiment, staged ablations, a human temporal-semantic audit, formal-closure stress tests, enlarged candidate pools, candidate-identifier shortcut auditing, a same-v5 backend replication, and a provenance audit, while explicitly stating the limits of closure, repair/CQ checks, candidate construction, and event-level evaluation.

2 Related Work

2.1 Ontology Evolution and Ontology Repair

Recent ontology-evolution research studies how evolving knowledge can be propagated through ontologies, dependent artifacts, and ontology alignments while preserving consistency and maintainability [3, 11]. OWL reasoners can verify logical consistency [16], but logical consistency does not guarantee continued agreement with the current normative source. Recent ontology-repair approaches address logical conflicts, axiom-level repair, and ontology-alignment repair using embedding-based, semantic-association, and probabilistic-logic techniques [8, 18]. SHACL-based repair provides a complementary line of work in which RDF constraint violations are detected and repaired through explicit constraint semantics [1, 2, 17]. Our setting differs from these approaches: the source ontology may remain logically consistent while one or more assertions are no longer supported by an evolving normative document.

2.2 Retrieval-Augmented Generation and Evidence Selection

Retrieval-augmented large language models use external knowledge sources to improve factuality, freshness, and evidence grounding in generation [4]. Recent work further suggests that evidence selection and reranking should consider not only relevance but also whether the selected evidence is sufficient and usable for downstream generation [14]. Our predicate-aware evidence focus adapts this principle to ontology repair from versioned normative documents.

2.3 Legal and Regulatory Information Extraction

Information extraction from legal and regulatory text must handle definitions, obligations, exceptions, conditions, priorities, and cross-sentence scope [12]. Legal AMR research demonstrates that structured legal semantics requires domain vocabulary, semantic roles, and complex relations [15]. Recent work on regulatory-rule extraction and legal-requirement translation has also explored structured rule representations and executable outputs [7, 13]. Our

goal is not rule extraction in isolation, but the use of structured semantics for finite-candidate ontology repair followed by formal OWL validation.

2.4 LLM-Assisted Ontology Engineering

Recent work has begun to investigate large language models across ontology-engineering activities, including requirements specification, ontology generation, evaluation, reuse, maintenance, and verification [9]. LLM-based ontology-generation methods have demonstrated the feasibility of producing OWL ontology drafts from user stories and competency questions [10]. More recent ontology-learning pipelines additionally combine LLM-based generation with ontology reuse, automated verification, and repair mechanisms [5]. These studies demonstrate the potential of language models to reduce the gap between natural-language requirements and formal ontology artifacts, while also exposing hallucination, structural instability, and verification challenges. We adopt a more conservative design: the LLM does not directly rewrite the ontology; instead, it first constructs evidence-grounded Semantic IR, after which symbolic candidate selection and formal validation constrain the final repair.

2.5 Neuro-Symbolic Reasoning and Ontology Repair

By “neuro-symbolic” we mean modular neural-symbolic integration rather than end-to-end differentiable theorem proving. Recent LLM-assisted ontology-engineering research shows that neural models can support semantic interpretation and structured ontology generation [5, 9, 10], while OWL semantics and reasoners provide explicit and inspectable symbolic constraints [16]. ECR-REPAIR uses Semantic IR as an interface contract between these two roles: neural components extract and refine evidence-grounded semantics and perform fallback entailment verification only under candidate ambiguity; symbolic components enforce typing, deterministic temporal normalization, constraint-aware ranking, and OWL/RDF consistency, graph-delta, and repair/CQ regression validation. The symbolic layer is therefore not merely a post-hoc safety mechanism; neural interpretations enter the decision process through an explicit and auditable intermediate representation from the outset.

2.6 Selective Prediction and Trustworthy Repair

Selective prediction allows a model to abstain when its prediction is insufficiently reliable, thereby trading coverage against selective risk. Recent NLP research has systematically evaluated confidence functions and evaluation methodology for neural selective prediction [6]. ECR-REPAIR operationalizes this principle for structured knowledge repair: the system fails closed when evidence is insufficient, Semantic IR is incomplete, candidate ranking is non-unique, or formal closure fails. Unlike work that evaluates complete risk-coverage curves, we report a single deployment operating point under frozen thresholds together with coverage, abstention, and selected-case semantic risk. We therefore claim an operationalization of selective-prediction principles in ontology repair, rather than a new theory of selective prediction.

3 Problem Formulation

Let D be a collection of versioned normative documents, O_0 a source ontology that may contain a semantic mismatch, and E a repair event. Each event contains public metadata including a subject, predicate, semantic type, case context, and evidence source. Let $D_E \subseteq D$ denote the evidence window provided for event E , and let M_E denote its public event metadata. The system is given a finite candidate set

$$C_E = \{c_1, c_2, \dots, c_k\}.$$

Each candidate contains a display value and a formal OWL/RDF edit operation. The private oracle candidate c_E^* is read only for offline scoring and is excluded from model inputs, semantic extraction or recovery, and candidate-decision prompts.

Meaning of the evidence-relative task. We treat D_E as the frozen event evidence available at deployment time, not as a complete historical archive. The immediate objective is therefore to determine whether the source assertion is incompatible with current evidence and to select a repair supported by that evidence from the provided candidates. A claim of strict historical drift requires independently verifiable D_{t_0} and D_{t_1} evidence. Stating this boundary at problem-formulation time prevents the later repair accuracy from being misread as the accuracy of historical-version reconstruction.

Running example. For intuition, consider an abstract example that follows the structure of an RFC version update but is not counted as a benchmark sample. Suppose the source ontology contains $a_0 = (s, \text{currentSpec}, X)$, while the current evidence D_{cur} explicitly states that “Y supersedes X” and identifies Y as the current specification. The candidate set contains “retain X,” “update to Y,” and a semantically adjacent distractor Z. The evidence-relative task requires only

$$\text{Support}(a_0, D_{\text{cur}}) \neq \text{supported},$$

$$\text{Support}((s, \text{currentSpec}, Y), D_{\text{cur}}) = \text{supported}.$$

Semantic IR should then record X as superseded and Y as current; candidate ranking should prefer “update to Y”; and formal closure should verify that the patch removes only the obsolete triple, adds the intended target triple, and preserves ontology consistency. The example illustrates the responsibility split in our design: language interpretation reconstructs semantics, the candidate layer makes the repair decision, and formal closure validates execution rather than assigning all responsibilities to a single component.

Conditional-task assumptions. We study *conditional repair decision making*, not complete end-to-end ontology maintenance. For each formal event: (i) the potential drift location and target subject-predicate are given; (ii) the relevant evidence source and evidence window are provided, and EvidenceFocus performs predicate-aware filtering only within that provided evidence rather than evaluating open-corpus retrieval recall; and (iii) the primary v5 benchmark uses three candidates and guarantees that one manually reviewed correct repair is present. Accordingly, we do not evaluate drift discovery, open-corpus evidence retrieval, or open-ended candidate generation. To test sensitivity to finite candidate-set size, we additionally derive $k = 5$ and $k = 10$ hard-negative variants without changing the parent benchmark. These variants support robustness

claims only for larger finite candidate pools, not for open-ended candidate generation or arbitrarily large search spaces.

The method outputs either a candidate $\hat{c}_E$ or ABSTAIN. A selected candidate counts as a successful repair closure only if all of the following hold:

- (1) the selected candidate matches the private oracle;
- (2) the candidate OWL patch is materialized successfully;
- (3) the repaired ontology passes reasoner consistency checking;
- (4) the actual RDF graph delta matches the edits declared by the candidate, with no additional undeclared triples; and
- (5) the frozen repair/CQ regression checks pass.

We do not claim to solve a globally minimum-edit repair problem over all possible repairs. “Minimal” or “restricted” change is operationally limited to requiring that the realized graph delta contain only the candidate’s declared target additions and deletions, without undeclared extra edits.

The primary metrics are closure accuracy, oracle-selection accuracy, and strict event success. To characterize the risk-coverage trade-off, we additionally report Coverage, Abstention, selected precision, and the Wrong-selected rate. Let

$$\text{Coverage} = \Pr(\text{Select}), \quad P_{\text{sel}} = \Pr(\text{Oracle} \mid \text{Select}),$$

so under binary semantic correctness of the selected candidate, the conditional semantic risk is

$$R_{\text{sem}} = \Pr(\hat{c}_E \neq c_E^* \mid \text{Select}) = 1 - P_{\text{sel}}.$$

All thresholds were frozen before the final benchmark, so these quantities describe one fixed operating point rather than a risk-coverage curve obtained by post-hoc threshold tuning.

4 Method

4.1 Neuro-Symbolic Repair Architecture

We call the final method ECR-REPAIR (Evidence-Constrained Semantic IR Repair) and position it as an evidence-constrained modular neuro-symbolic ontology repair framework. To preserve traceability between the manuscript, code, and experimental logs, we give the implementation identifiers once here: Base Semantic IR Pipeline (M13), Clause-Level Semantic Recovery (M14), Temporal Anchor Recovery (M15), Constraint-Aware Candidate Ranking and Selective Gate (V4.3), and Fallback Candidate Entailment Verifier (M16). Functionally, the framework contains four roles: a neural semantic interpretation layer $\mathcal{N}_{\text{sem}}$, a symbolic semantic decision layer $\mathcal{S}_{\text{decision}}$, an on-demand neural candidate-verification layer $\mathcal{N}_{\text{verify}}$, and a formal symbolic closure layer $\mathcal{S}_{\text{formal}}$:

$$\mathcal{N}_{\text{sem}} \rightarrow \mathcal{S}_{\text{decision}} \rightarrow [\mathcal{N}_{\text{verify}}] \rightarrow \mathcal{S}_{\text{formal}}.$$

$\mathcal{N}_{\text{sem}}$ consists of Predicate-aware Evidence Focus, the Base Semantic IR Pipeline, and Clause-Level Semantic Recovery when needed. $\mathcal{S}_{\text{decision}}$ contains Typed Semantic IR, Temporal Anchor Recovery, and the Constraint-Aware Candidate Ranking and Selective Gate. $\mathcal{N}_{\text{verify}}$ invokes the Fallback Candidate Entailment Verifier only when the symbolic ranking is ambiguous. $\mathcal{S}_{\text{formal}}$ performs formal validation through OWL/RDF patching, a reasoner, RDF graph-delta verification, and frozen repair/CQ regression checks. This design separates responsibility for neural interpretation, symbolic

constraint, neural fallback verification, and formal checking. Informally, the first two layers answer “what does the evidence say, and which candidate is compatible with it?”; the latter two answer “is the residual ambiguity resolvable, and can the patch be executed safely?”

The ranking stage first defines the maximizing candidate set

$$\mathcal{M}_E = \arg \max_{c_i \in C_E} S(c_i \mid \text{IR}, F; \tau_E),$$

where τ_E denotes the semantic type of event E . If $\mathcal{M}_E$ contains exactly one candidate and that candidate satisfies the frozen gate, then

$$c_{\text{rank}} = \begin{cases} c, & \mathcal{M}_E = \{c\} \text{ and } c \text{ is admissible,} \\ \perp, & \text{otherwise.} \end{cases}$$

If ranking cannot produce a unique admissible candidate, the Fallback Candidate Entailment Verifier evaluates candidates independently against the evidence:

$$c_{\text{verify}} = \begin{cases} c_j, & \text{if exactly one } c_j \in C_E \text{ is supported} \\ & \text{with high/medium confidence,} \\ \perp, & \text{otherwise.} \end{cases}$$

The pre-closure decision is

$$\tilde{c}_E = \begin{cases} c_{\text{rank}}, & c_{\text{rank}} \neq \perp, \\ c_{\text{verify}}, & c_{\text{rank}} = \perp \wedge c_{\text{verify}} \neq \perp, \\ \text{ABSTAIN}, & \text{otherwise,} \end{cases}$$

and the final output is

$$\hat{c}_E = \begin{cases} \tilde{c}_E, & \tilde{c}_E \neq \text{ABSTAIN} \wedge \text{OWLClosure}(\tilde{c}_E) = \text{true}, \\ \text{ABSTAIN}, & \text{otherwise.} \end{cases}$$

4.2 Candidate-Content-Isolated Neural Interpretation and Semantic IR

The Base Semantic IR Pipeline is the backbone of the neural semantic interpretation layer rather than a prompt that directly chooses an answer. Its processing chain is **predicate-aware evidence focus** $\rightarrow$ **task formulation** $\rightarrow$ **atomic fact extraction** $\rightarrow$ **evidence-grounded rule refinement** $\rightarrow$ **robust IR normalization**. EvidenceFocus uses the public subject, predicate, and case context to retain the most predicate-relevant spans within the provided evidence window. Task formulation selects the appropriate field constraints for Temporal-Version, General Rule-Exception, or Cross-Sentence Scope from the public semantic type and predicate. Atomic extraction requires facts to be linked to evidence spans rather than producing unsupported conclusions. An LLM then performs evidence-grounded critique and refinement of the draft facts and converts them into typed, candidate-content-isolated SEMANTIC IR. The neural component therefore compresses natural-language evidence into traceable semantics but does not own the final candidate decision; its output must pass through the subsequent symbolic layers before it can become an ontology repair.

Robust IR normalization handles structural compatibility only, such as field aliases, normalizable scalars, and schema-compatible representations. It does not read candidates or use candidate values to fill missing semantics. If evidence cannot supply a type-required critical slot, the IR is marked incomplete and routed to Clause-Level

ECR-Repair: Evidence-Constrained Neuro-Symbolic Ontology Repair

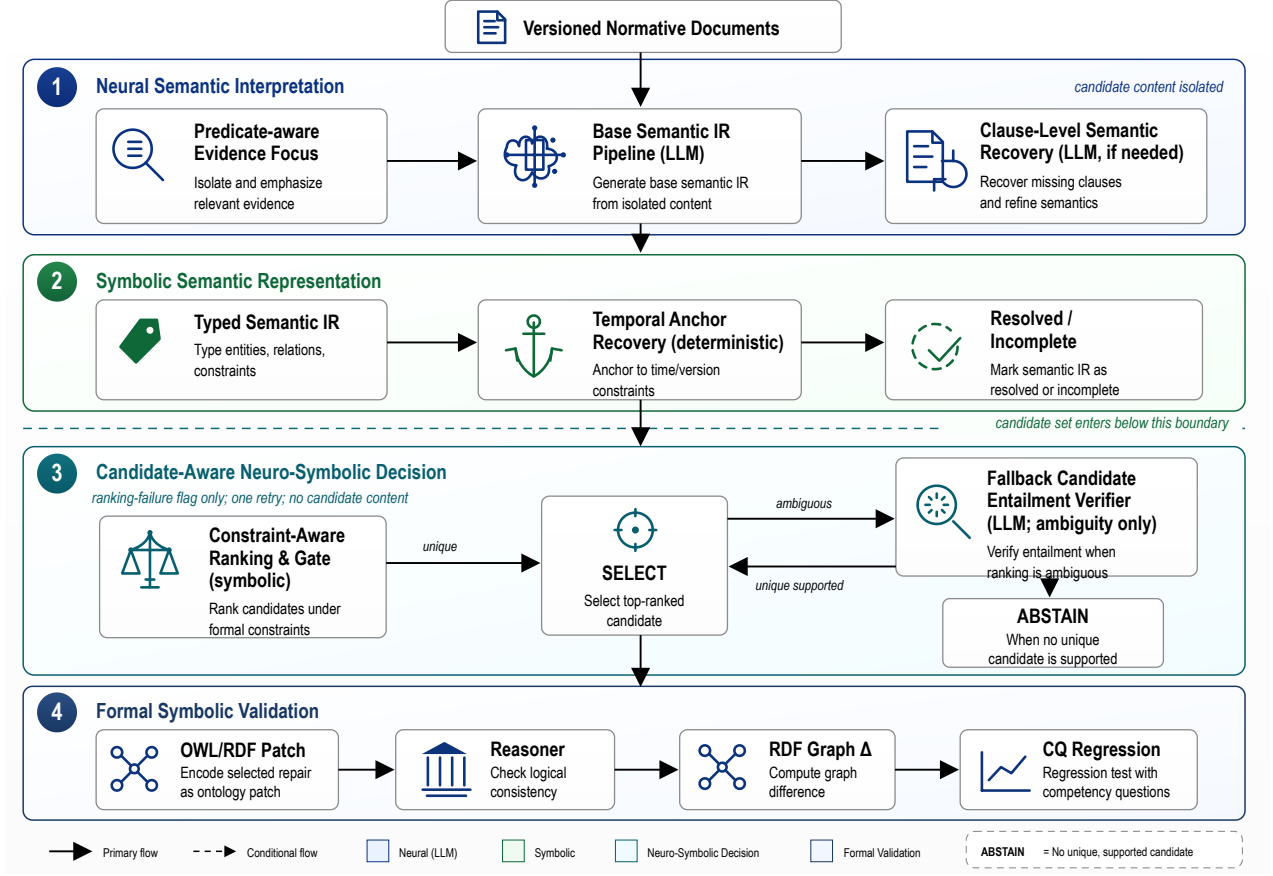


Figure 1: Overall evidence-constrained neuro-symbolic architecture of ECR-REPAIR. Semantic IR is the neural-to-symbolic interface; the candidate set becomes visible only in the candidate-aware decision layer. The Fallback Candidate Entailment Verifier is triggered only when symbolic ranking is ambiguous, and the dashed feedback path carries only a content-free ranking-failure flag.

Table 1: Evidence-constrained ontology repair algorithm

Step	Operation
1	$F \leftarrow \text{EvidenceFocus}(D_E, M_E)$ using only public documents, event metadata, and the provided evidence window.
2	$\text{IR} \leftarrow \text{BaseIR}(F, M_E)$ to produce candidate-content-isolated Semantic IR.
3	If the General Rule-Exception or Cross-Sentence Scope IR is incomplete, set $\text{IR} \leftarrow \text{ClauseRecovery}(\text{IR}, F, M_E)$; if Temporal-Version anchors are unresolved, set $\text{IR} \leftarrow \text{TemporalRecovery}(\text{IR}, F, M_E)$.
4	After IR is fixed, reveal C_E , compute $S_i = S(c_i \mid \text{IR}, F; \tau_E)$, and return c_{rank} or $\text{ranking_failed}=\text{true}$.
5	For General Rule-Exception or Cross-Sentence Scope, a $\text{ranking_failed}=\text{true}$ result may trigger one additional candidate-content-isolated Clause-Level Semantic Recovery pass; the retry receives only the Boolean flag and then repeats Step 4.
6	If ranking is still non-unique, invoke the Fallback Candidate Entailment Verifier and obtain c_{verify} or ABSTAIN.
7	Execute the selected OWL patch, reasoner check, RDF graph-delta validation, and frozen repair/CQ regression checks; return ABSTAIN if any stage fails.
8	Return the closure-validated repair $\hat{c}_E$.

Semantic Recovery, Temporal Anchor Recovery, or final fail-closed behavior. The Base Semantic IR Pipeline is therefore designed to

answer “what does the evidence say?” rather than “which candidate looks most like the answer?”

4.3 Three Semantic Event Types

Temporal-Version represents version changes, effective dates, supersession relations, or changes in the currently valid state. Its IR contains fields such as subject, old value, new value, effective time, relation, current value, and evidence.

General Rule-Exception represents a general rule overridden by an exception, condition, or priority relation. Its IR contains subject, general rule, exception condition, exception rule, priority, result, and evidence.

Cross-Sentence Scope represents a qualifier or scope relation attached across sentence boundaries. Its IR contains subject, statement, qualifier, scope target, scope relation, result, and evidence.

4.4 Candidate-Content Isolation Protocol

Semantic extraction and recovery modules cannot read candidate values, operations, identifiers, or scores. Top- k identities, the private oracle, and manually authored formal policy are also hidden. Initial Semantic IR is formed while candidate content is invisible. The only signal allowed to travel from the candidate-decision layer back to semantic recovery is the Boolean control flag `ranking_failed=true`. It means only “perform one more recovery pass” and conveys no information about which candidate is leading, trailing, or conflicting. We therefore use the term *candidate-content isolation*, rather than claiming absolute independence from the existence of the candidate set.

4.5 Clause-Level Semantic Recovery

Clause-Level Semantic Recovery handles two cases: (i) structurally incomplete General Rule-Exception or Cross-Sentence Scope IR after the Base Semantic IR Pipeline, and (ii) General Rule-Exception or Cross-Sentence Scope cases whose IR is rankable but cannot be uniquely decided by the Constraint-Aware Candidate Ranking and Selective Gate. The first case is recovered while candidate content remains invisible. The second receives only `ranking_failed=true`, without candidate identifiers, values, scores, top- k identities, or candidate-derived semantics. For General Rule-Exception, the recovery step separates the general rule, exception trigger, exception conclusion, and priority into distinct clauses. For Cross-Sentence Scope, it separates the qualifier, qualified statement, and scope attachment. Processing continues only if recovery yields a complete IR that passes subsequent gating; otherwise the system remains abstinent.

4.6 Temporal Anchor Recovery

Temporal Anchor Recovery addresses temporal-anchor ambiguity identified during development. Versioned specifications may mention publication dates, references to publication dates, effective dates, superseded versions, the currently valid version, and historical references in the same evidence window. Treating the most recent date or the last-mentioned version identifier as the current value can therefore create systematic errors. Temporal Anchor Recovery reads no candidates and uses no candidate score or margin. It normalizes public evidence and document-time metadata into role-bearing anchors; only afterward does the Constraint-Aware Candidate Ranking and Selective Gate map the fixed temporal IR to candidates.

Temporal Anchor Recovery is not a single-label classifier but a deterministic **role-relation normalization** procedure. Let $\mathcal{V}_E$ denote the version/specification entities appearing in event E . Then

$$\text{NormalizeTemporal}(\text{Evidence}_E, \text{Metadata}_E) = (\mathcal{R}_E, \mathcal{T}_E, u_E),$$

where

$$\mathcal{R}_E \subseteq \mathcal{V}_E \times \{\text{current, effective, superseded, reference, publication}\},$$

$$\mathcal{T}_E \subseteq \mathcal{V}_E \times \{\text{supersedes, updates, updatedBy}\} \times \mathcal{V}_E,$$

$$u_E \in \{\text{resolved, unresolved}\}.$$

The same entity may therefore carry both current and effective roles, while supersedes/updates directions are represented independently rather than mixed into role labels. Temporal Anchor Recovery fails closed only when $u_E = \text{unresolved}$ and the public evidence cannot resolve the ambiguity further. The rules in Table 2 concern roles, relations, and direction; they do not use candidate scores. The later ranking and gate, not Temporal Anchor Recovery, owns `min_score`, `min_margin`, and `temporal unique-top1`.

Continuation of the running example. Suppose the $X \rightarrow Y$ replacement example in Section 3 additionally contains “RFC X published in year t_1 ,” “RFC Y obsoletes RFC X,” and “this specification takes effect at t_2 .” Temporal Anchor Recovery can produce

$$\mathcal{R}_E \supseteq \{(X, \text{publication}), (X, \text{superseded}), \\ (Y, \text{current}), (Y, \text{effective})\},$$

$$(Y, \text{supersedes}, X) \in \mathcal{T}_E.$$

This example is illustrative only and is not an additional v5 benchmark sample. The candidate set remains hidden until this temporal IR has been fixed.

4.7 Constraint-Aware Candidate Ranking and Selective Gate

The Constraint-Aware Candidate Ranking and Selective Gate is a deterministic mapping from fixed SEMANTIC IR to the finite candidate set. To avoid reducing the implementation to an opaque single similarity score, the scorer retains inspectable subcomponents for every candidate:

$$\phi(c_i, \text{IR}, F) = (s_{\text{tok}}, s_{\text{exact}}, \\ s_{\text{family}}, s_{\text{code}}, s_{\text{num}}, s_{\text{sem}}, p_{\text{neg}}).$$

The first five components capture token overlap, exact match, family match, code match, and numeric/date match. s_{sem} is a type-specific semantic-consistency bonus, and p_{neg} is a penalty for negation, directional conflict, or semantic inconsistency. The aggregate score is

$$S(c_i \mid \text{IR}, F; \tau_E) = \text{Agg}(\phi(c_i, \text{IR}, F), \tau_E),$$

where Agg is the deterministic aggregate used in the frozen implementation and τ_E encodes semantic-type constraints. No scorer parameters are re-estimated on v5; both code-level aggregation logic and thresholds were fixed at method freeze. In contrast to retaining only an aggregate score, these subcomponents provide an audit trail for why a candidate scored highly, why it was penalized for direction or negation, and why a gate fired. Robust IR only normalizes field aliases and compatible structural forms and never uses candidate content to “fill in” an answer.

Table 2: Candidate-content-isolated temporal-anchor rules used by Temporal Anchor Recovery

Evidence/metadata pattern	Normalized role	Processing principle
Explicit "obsoletes/supersedes" relation	role: superseded/current; relation: supersedes	Mark the superseded object with a historical role and record the direction of the replacement relation separately rather than collapsing it into a single label.
Explicit "updates/updated by" relation	relation: updates/updatedBy	Preserve update direction without equating update with complete replacement; current status is determined separately from textual evidence.
Explicit effective date / "takes effect"	role: effective	Bind the effective role to the relevant specification entity; publication remains a distinct role and does not overwrite effective.
Explicit "current/this specification" plus historical reference	role: current/reference	Permit multiple roles for the same entity and keep current statements separate from historical references.
Explicit publication date	role: publication	Preserve publication as document-time metadata without inferring current status from publication alone.
Conflicting dates or versions	resolution: unresolved	Do not guess from position or the numerically latest value; fail closed if public textual relations cannot resolve the conflict.

The gate thresholds are frozen. The default minimum score is $s_{\min} = 0.30$; General Rule–Exception and Cross-Sentence Scope use $s_{\min} = 0.24$. The minimum score margin is fixed at $m_{\min} = 0.00$.

Temporal-Version additionally uses a temporal unique-top1 condition. The top candidate may continue only when role and relation direction are unique, no explicit conflict exists, and no admissible tie remains. Otherwise the system returns `ranking_failed=true`. That signal may trigger at most one Clause-Level Semantic Recovery pass and exposes no candidate feature, identity, or lead information.

Selective gate. The purpose of the gate is not to maximize the number of repairs but to reject a repair when evidence, Semantic IR, or the candidate decision is insufficiently determined. The system fails closed if IR is incomplete, JSON is invalid, a candidate score falls below its frozen threshold, the candidates cannot be distinguished uniquely, or the subsequent Fallback Candidate Entailment Verifier cannot identify exactly one supported candidate. The gate controls whether a candidate is eligible to proceed; it does not equate formal consistency with semantic correctness.

4.8 Fallback Candidate Entailment Verifier

The Fallback Candidate Entailment Verifier is triggered only when the preceding Constraint-Aware Candidate Ranking and Selective Gate cannot make a unique decision. The main v5 experiment uses the DeepSeek API model identifier `deepseek-chat`, with temperature 0, a 500-token output limit, and a 180-second timeout. The API does not provide strict seed control, so seed is retained only as an audit field.

The verifier calls the model independently for each candidate rather than presenting all three candidates in one comparative prompt. Each call can see the temporary candidate identifier, display value, and operation; the identifier is used only to align the output. The verifier labels a candidate as supported, contradicted, unrelated, or underspecified and reports categorical confidence as

high, medium, or low. This confidence is model-reported categorical confidence, not a calibrated probability or posterior probability.

A candidate proceeds to formal closure if and only if exactly one candidate is labeled supported with high or medium confidence. All other outcomes cause abstention. Because `deepseek-chat` is a service-side model alias that may change over time, exact reproduction requires the run date, raw outputs, and configuration/freeze manifest in addition to the model name.

4.9 Formal OWL/RDF Closure and Repair/CQ Regression

Each candidate operation is represented as a finite RDF/OWL edit set

$$\Delta_c = (\Delta_c^-, \Delta_c^+),$$

where Δ_c^- is the candidate-declared deletion set and Δ_c^+ the addition set, each containing target RDF/OWL triples or statements.

For the ontology O_c produced by executing candidate c , the realized graph delta is

$$\Delta_{\text{graph}}(O_0, O_c) = (O_0 \setminus O_c, O_c \setminus O_0).$$

Formal closure requires

$$\Delta_{\text{graph}}(O_0, O_c) = \Delta_c.$$

Closure fails if undeclared changes appear or if declared edits are not realized as expected. This condition controls patch side effects; it does not solve for a globally minimum edit distance across all possible repairs.

We refer to the final regression interface uniformly as **frozen repair/CQ regression checks**. The frozen implementation is dominated by event-specific triple-level repair checks while retaining a competency-question-compatible regression interface. These checks share the repair-event construction protocol and are not generated or adapted from model outputs during formal runs. They therefore validate declared graph effects, logical consistency, and predefined regression behavior of the selected patch, but they are

Table 3: Summary of the final blind-large benchmark

Property	Value
Events	264
Runs per event	5
Total runs	1320
Domains	11
Included source families	13
Candidate-source registry	69
Semantic types	3
Candidate rows	792
Candidate-note leakage rows after sanitization	0
Benchmark manifest SHA-256	40fcc2c3...9bb0e038

not an independent external semantic gold set and do not act as a semantic oracle. Formal closure consists of OWL/RDF materialization, reasoner consistency, RDF graph-delta verification, and frozen repair/CQ regression checks; failure at any stage returns ABSTAIN. RQ3 directly stress-tests this capability boundary.

5 Benchmark and Evaluation Protocol

5.1 Benchmark Dataset

The primary experiment uses external-real-holdout-v5-blind-large. This benchmark was newly constructed from public RFC normative documents after the complete method had been frozen. It contains 264 manually reviewed repair events, each evaluated over five runs, for 1320 runs in total. The data cover 11 domains, 13 RFC source families represented in the final set, and three semantic event types; the candidate-source registry contains 69 public source records. Earlier v1/v3/v4 datasets were used for diagnosis, development, calibration, or supplementary validation and are not the source of the main result.

5.2 Method Freeze and Blind-Evaluation Protocol

The method-freeze manifest has SHA-256 prefix/suffix e27619e2...5def068 and records the fixed modules, thresholds, input-isolation rules, and no-tuning policy. v5 was constructed afterward, followed by oracle/support review for all 264 events, candidate-note sanitization, structural validation, deeper quality auditing, and benchmark freeze. Sanitization changed candidate notes only; it did not change candidate identifiers, display values, operations, or the oracle, and the frozen method never reads notes. In this paper, **blind** has an operational meaning: the complete method was frozen before v5 construction, the private oracle was isolated from inference inputs, and no post-hoc method tuning was performed after v5 results were observed. The term does not imply third-party double-blind benchmark annotation.

5.3 Candidate Construction and Identifier Regularity

Each v5 event contains exactly three candidates and exactly one manually reviewed oracle candidate. The oracle is constructed

from the semantic mismatch between the public evidence and the source-OWL assertion requiring repair and represents the repair supported by current normative evidence. One distractor typically preserves the old version, obsolete value, or unrepaired semantics; the other is an executable semantic neighbor that is not best supported by the current evidence. All candidate operations must be executable, all OWL patches must be materializable, display values must not contain explicit answer markers, and candidate notes are excluded from method input and sanitized for leakage. **Identifier regularity:** the frozen v5 construction template assigns the oracle to CAND_002 for all 264 events, while CAND_001 and CAND_003 are distractors. This regularity comes from benchmark construction rather than private-oracle exposure, but it creates a potential identifier shortcut. We therefore perform a separate per-event candidate-ID permutation audit. Because the correct repair is guaranteed to occur in the finite candidate set, the results must not be interpreted as open-ended repair-generation performance.

5.4 Evaluation Scope and Experimental Unit

The experimental unit is an **event-level OWL/RDF patch**, not total ontology size. Candidate operations are normalized into finite add/delete edit sets and executed around the target subject/predicate assertion and its direct semantic context. We therefore evaluate whether the method can select and execute the correct local repair given a known potential drift event. We do not vary large shared-ontology size, TBox depth, or whole-knowledge-base reasoning time, and make no claim about reasoning scalability for very large ontologies.

5.5 Compared Methods and Baselines

We use staged ablations: Base Semantic IR Pipeline; Base + Clause Recovery; Base + Clause + Temporal Recovery; and full ECR-REPAIR with the Fallback Candidate Entailment Verifier. We also use Direct LLM (Candidate Value) and Direct LLM (Value + Formal Operation) as candidate-selection baselines.

5.6 Metrics and Statistical Analysis

The five runs of a single event share the same event-level semantic structure, so we do not treat all 1320 attempts as independent observations. Attempt-level accuracy is reported descriptively, with uncertainty estimated by an **event-clustered bootstrap**: the 264 events are the resampling units and all five runs for a sampled event are carried together. We use 10,000 percentile-bootstrap replicates. Strict event success is already event-level and is therefore accompanied by a Wilson 95% interval. Stage differences use event-clustered paired bootstrap as well. We do not use attempt-level McNemar p values as primary or supplementary inference, avoiding pseudo-replication from the five correlated runs per event.

5.7 Implementation and Reproducibility Settings

Implementation and compute fairness. Direct LLM (Candidate Value) and Direct LLM (Value + Formal Operation) answer the effectiveness question “how well can a model do when candidate information is exposed directly?” They are not compute-matched to ECR-REPAIR: the full pipeline may perform a Clause-Level Semantic

Benchmark and Method Freeze Timeline

Chronological workflow from dataset development to final evaluation

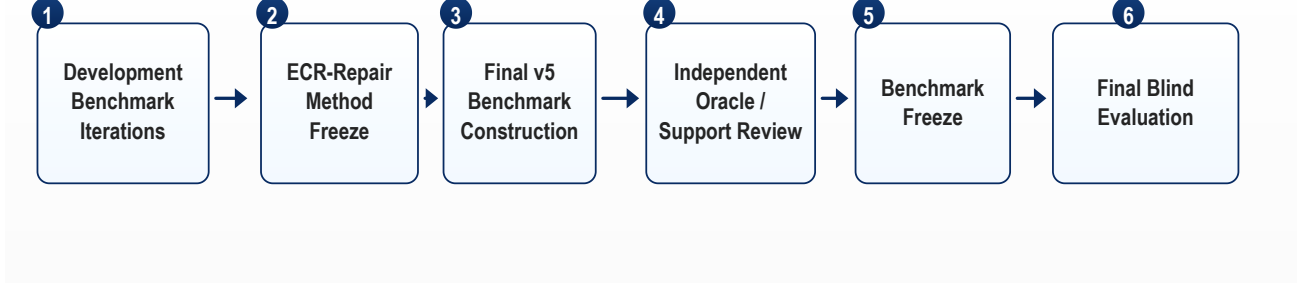


Figure 2: Timeline of method freeze, benchmark construction, and final blind evaluation.

Table 4: Main v5 implementation configuration for ECR-REPAIR

Stage	Backend/execution	Main parameters	Candidate content visible?
Base Semantic IR Pipeline	DeepSeek API deepseek-chat	temp=0, max tokens=900, timeout=180s	No
Clause-Level Semantic Recovery	DeepSeek API deepseek-chat	two JSON calls, 700 tokens each, timeout=180s; at most one retry	No; Boolean failure flag only
Temporal Anchor Recovery	Deterministic recovery	temporal-role normalization; no candidate scores	No
Constraint-Aware Ranking/Gate	Constraint-aware scorer	min score=0.30; General Rule-Exception / Cross-Sentence Scope min score=0.24; min margin=0.00; temporal unique-top1; robust IR on	Yes
Fallback Entailment Verifier	DeepSeek API deepseek-chat	per-candidate, temp=0, max tokens=500, timeout=180s	Yes
OWL closure	RDF/OWL materialization + reasoner + repair/CQ checks	graph delta, consistency, frozen repair/CQ regression	Selected candidate only

Recovery retry or per-candidate fallback-verifier calls, whereas the simple baselines use shorter call structures. We therefore do not use these baselines to claim better token or latency efficiency. The cloud deepseek-chat alias does not guarantee immutable future weights, so reproducibility requires the run date, raw responses, and configuration manifest. The manuscript fixes the logical closure interface as materialization-reasoner-graph-delta-repair/CQ regression but does not invent unverified software-version metadata for the specific reasoner/RDF library/repair-check harness. A formal artifact release should expose those environment and dependency details.

6 Evaluation Results

6.1 RQ1: How Effective Is ECR-Repair Overall?

Main result. RQ1 first asks whether the frozen method can complete local repairs correctly and safely on the final v5 benchmark.

Table 5 summarizes closure, oracle-selection, and strict-event results for the three semantic types.

The main result uses frozen v5, the DeepSeek backend, and the frozen full method. The 1320 runs come from five executions of each of 264 events; attempt-level proportions are descriptive, while event-level uncertainty follows the clustered protocol in Section 5.6. Table 5 shows that the remaining failure is not an incorrect patch but a conservative abstention: the only failed run is one General Rule-Exception run of H5_E017. The final wrong-selected count is zero.

Direct-candidate baselines. Table 6 asks whether the high final accuracy can be explained simply by an easy three-candidate task.

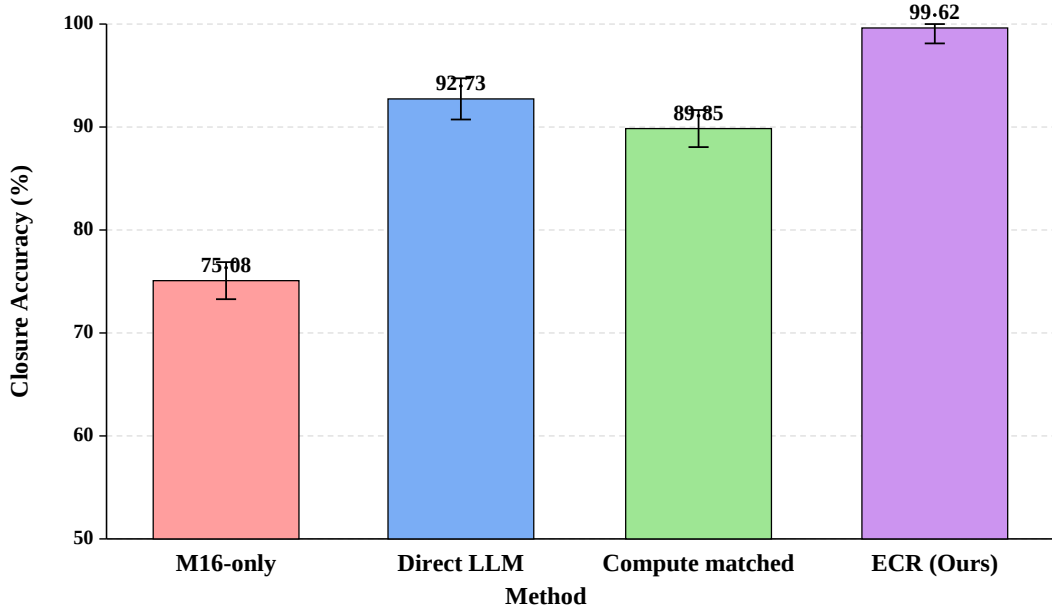
The candidate-value baseline already reaches 93.48%, so we do not portray v5 as a task in which the candidates are intrinsically difficult to distinguish. Direct LLM (Candidate Value) has an oracle-selection failure rate of 86/1320 (6.52%), whereas ECR-REPAIR fails on only 1/1320 (0.08%), and that single failure is abstention

Table 5: Main results of ECR-REPAIR on the final frozen v5 blind-large benchmark

Semantic type	Closure	Oracle	Strict event
ALL	1315/1320 (99.62%)	1315/1320 (99.62%)	262/264 (99.24%)
Temporal-Version	440/440 (100.00%)	440/440 (100.00%)	88/88 (100.00%)
General Rule-Exception	439/440 (99.77%)	439/440 (99.77%)	87/88 (98.86%)
Cross-Sentence Scope	440/440 (100.00%)	440/440 (100.00%)	88/88 (100.00%)

Table 6: Full candidate-selection baselines compared with ECR-REPAIR

Method	Oracle selection	OWL closure	Notes
Direct LLM (Candidate Value)	1234/1320 (93.48%)	–	evidence + candidate value
Direct LLM (Value + Formal Operation)	1171/1320 (88.71%)	–	evidence + formal operation
Base Semantic IR Pipeline	1022/1320 (77.42%)	1022/1320 (77.42%)	base Semantic IR repair pipeline
ECR-REPAIR	1315/1320 (99.62%)	1315/1320 (99.62%)	complete closure-enabled repair

**Figure X.** Comparison with strong LLM baselines on closure accuracy (%). Error bars indicate 95% bootstrap confidence intervals over events.**Figure 3: Comparison with strong baselines. ECR-Repair improves closure accuracy over direct LLM and compute-matched baselines while preserving formal validation and candidate selection constraints.**

rather than wrong selection. The relevant contribution is therefore residual-error compression and fail-closed risk control on top of a strong direct baseline.

Uncertainty and selective repair. The stagewise gains and uncertainty estimates are summarized in Tables 7 and 8, while Table 9 reports the corresponding selective-repair operating point.

6.2 RQ2: Which Components Contribute, and Does Temporal Recovery Improve the Target Semantics?

Staged ablation. Table 10 measures the incremental contribution of each stage, with particular attention to whether Temporal Anchor Recovery accounts for the largest gain.

Temporal Anchor Recovery is the largest source of stagewise gain. Because final candidate accuracy does not by itself establish that the temporal representation is correct, we additionally audit

Table 7: Event-clustered paired-bootstrap estimates for stage gains

Comparison	Mean gain	Event-clustered 95% CI
ECR-REPAIR vs Base Semantic IR Pipeline	22.50 pp	17.73–27.42 pp
Base + Clause Recovery vs Base Semantic IR Pipeline	2.73 pp	1.06–4.70 pp
Base + Clause + Temporal Recovery vs Base + Clause Recovery	18.71 pp	14.32–23.41 pp
ECR-REPAIR vs Base + Clause + Temporal Recovery	1.06 pp	0.00–2.42 pp

Table 8: Uncertainty intervals and independent units for the main result

Metric	Point estimate	95% interval	Independent/resampling unit
Attempt closure	1315/1320 (99.62%)	99.77–100.00%	264 events, retaining 5 runs/event
Strict event success	262/264 (99.24%)	97.89–99.93%	264 events (Wilson)

Table 9: Coverage, abstention, and selected precision

Method	Coverage	Abstention	Pr(Oracle Select)	Wrong selected
Base Semantic IR Pipeline	1034/1320 (78.33%)	286/1320 (21.67%)	1022/1034 (98.84%)	12/1320 (0.91%)
ECR-REPAIR	1315/1320 (99.62%)	1/1320 (0.08%)	1319/1319 (100.00%)	0/1320 (0.00%)

Table 10: Staged ablation results

Method stage	Closure	Strict event	Gain over Base	Stage gain
Base Semantic IR Pipeline	1022/1320 (77.42%)	195/264 (73.86%)	0.00 pp	–
Base + Clause Recovery	1058/1320 (80.15%)	205/264 (77.65%)	2.73 pp	2.73 pp
Base + Clause + Temporal Recovery	1305/1320 (98.86%)	261/264 (98.86%)	21.44 pp	18.71 pp
ECR-REPAIR	1315/1320 (99.62%)	262/264 (99.24%)	22.50 pp	1.06 pp

Temporal Anchor Recovery against manually constructed temporal gold annotations.

Human semantic audit of Temporal Anchor Recovery. For all 88 Temporal-Version events, an event-level temporal gold annotation was created without reading the private oracle, candidate answers, or M13/M15 outputs. Pairing this gold with the five formal v5 runs yields 440 descriptive attempts. Table 11 shows that Temporal Anchor Recovery primarily improves task-relevant role and effective-anchor normalization rather than every aspect of temporal relation extraction.

Across the 440 attempts, M15 improves 160 role exact-frame cases, with the event-clustered mean rising from 0.023 to 0.386. At the same time, relation-type accuracy decreases by 18.0 percentage points. We therefore interpret M15 as **task-oriented temporal-anchor normalization**, not as a complete temporal information-extraction module. In particular, 100% strict success on Temporal-Version repairs does not imply complete reconstruction of every temporal relation; the narrower conclusion is that sufficient structured semantics can support correct local repair.

6.3 RQ3: What Safety Guarantees Does Formal Closure Provide?

Formal-closure stress test. Table 12 asks a question that is deliberately separate from semantic-selection accuracy: which error classes can formal closure reject, and which responsibility lies outside its scope?

To separate “semantic candidate correctness” from “formal patch safety,” we generate five deterministic stress variants for every v5 event, yielding 1320 cases. This experiment does not call an LLM; it directly tests the frozen closure interface.

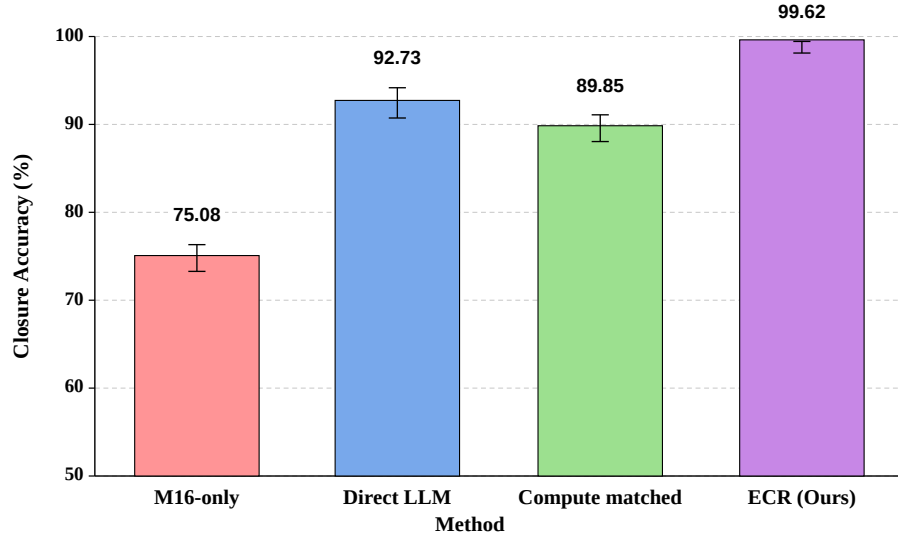
All valid control patches are accepted. Among the four non-oracle stress classes, closure rejects all cases in the three structural/logical/regression-violation classes, but it rejects none of the 264 semantically wrong yet formally self-consistent patches. Its capability boundary is therefore more accurately written as

$$\text{FormalClosureValid}(c) \Rightarrow \text{SemanticCorrect}(c).$$

Passing formal closure does not imply semantic correctness under the normative evidence. Formal closure should therefore be interpreted as a structural, logical, declared-graph-effect, and regression-safety layer *after* semantic candidate selection, not as an independent semantic oracle.

Table 11: Base IR versus Temporal Anchor Recovery on human temporal gold

Metric	Base IR	+ Temporal Anchor Recovery	Δ
Role exact-frame	2.3%	38.6%	+36.4 pp
Complete temporal frame	2.3%	38.6%	+36.4 pp
Effective anchor accuracy	8.0%	100.0%	+92.0 pp
Current anchor accuracy	100.0%	100.0%	0.0 pp
Role micro-F1	0.682	0.829	+0.147
Role macro-F1	0.628	0.700	+0.072
Relation type accuracy	62.3%	44.3%	-18.0 pp

**Figure X.** Comparison with strong LLM baselines on closure accuracy (%). Error bars indicate 95% bootstrap confidence intervals over events.**Figure 4: Temporal anchor recovery improves effective-anchor fidelity. The module improves task-oriented temporal normalization rather than attempting complete temporal relation extraction.****Table 12: Formal closure stress test (264 events $\times$ 5 variants)**

Variant	Meaning	Detection/acceptance	Main stage
VALID_ORACLE_CONTROL	correct oracle patch	264/264 accepted	final closure pass
EXTRA_UNDECLARED_EDIT	undeclared graph edit	264/264 rejected	graph delta
INCONSISTENT_PATCH	logically inconsistent patch	264/264 rejected	reasoner
REPAIR_CHECK_BREAK	frozen repair regression violated	264/264 rejected	repair check
WRONG_SEMANTIC_PATCH	non-oracle but triple/logically self-consistent	0/264 rejected	264/264 pass closure

6.4 RQ4: Is the Method Robust to Candidate-Set Size, Backend Choice, and Candidate Identifiers?

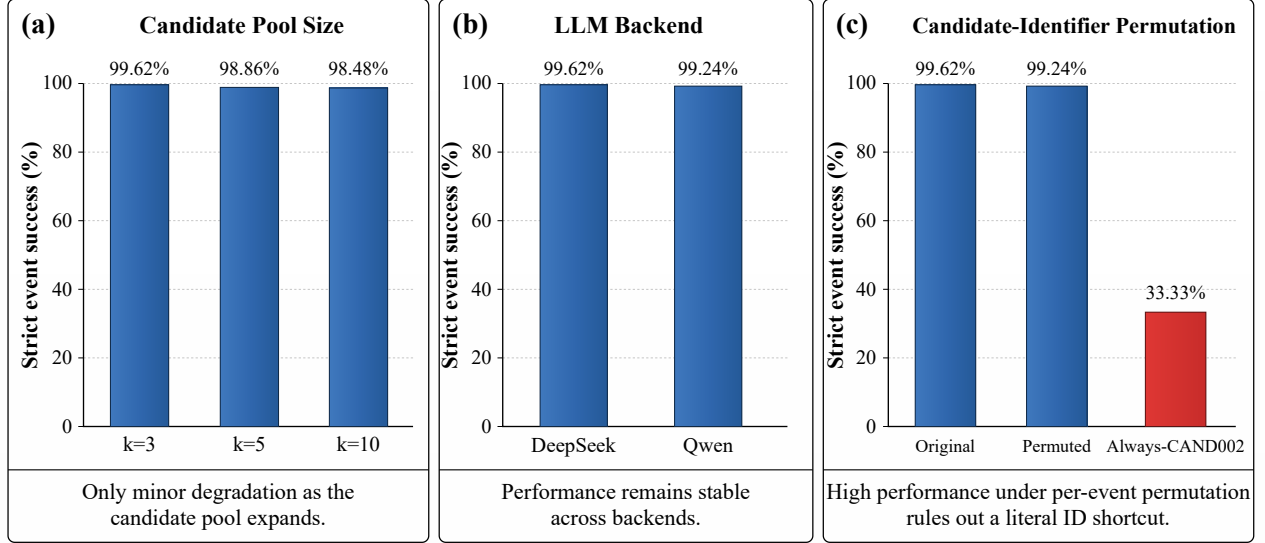
Candidate-pool size. Table 13 tests whether the frozen method remains stable as the finite candidate pool grows from $k = 3$ to $k = 5$ and $k = 10$. The parent events, evidence, oracle semantics, and frozen method are unchanged; thresholds, prompts, and gates are not retuned. Each condition uses 264 events $\times$ 5 runs.

Increasing the pool from three to ten candidates reduces strict event success by only three events (about 1.1 percentage points); Temporal-Version remains 88/88 throughout, while most degradation appears in General Rule-Exception. This supports stability under larger finite hard-negative pools but does not generalize to open-ended candidate generation or unbounded retrieval.

Same-v5 backend robustness. Table 14 tests whether the method depends critically on one cloud LLM backend. On the same frozen

Robustness Analysis of ECR-Repair

Metric: strict event success (%)



ECR-Repair remains robust to candidate-pool expansion, backend substitution, and candidate-ID permutation, while a literal CAND002 shortcut fails badly.

Figure 5: Robustness analysis of ECR-REPAIR under candidate-pool expansion, LLM backend substitution, and candidate-identifier permutation. Results are reported using strict event success.

Table 13: Candidate-pool size robustness

Pool	Closure	Strict events	Temporal	Rule-Exception	Cross-Sentence
$k = 3$	99.62%	262/264 (99.24%)	88/88	87/88	88/88
$k = 5$	99.39%	261/264 (98.86%)	88/88	86/88	87/88
$k = 10$	99.09%	260/264 (98.48%)	88/88	85/88	87/88

v5 benchmark with $k = 3$, DeepSeek is replaced by local Ollama qwen3.5:9b while the method logic, M15, ranking thresholds, candidate set, and closure remain unchanged. Each backend is evaluated over 264×5 runs.

Qwen differs from DeepSeek by one strict event. The Qwen run contains 15/1320 (1.1%) Base IR timeouts concentrated in a few events; all are retained as failures and none are selectively rerun. This supports backend robustness and portability, not model independence.

Candidate-identifier shortcut audit. Table 15 directly tests whether the v5 construction regularity in which every oracle is CAND_002 could explain the result through a literal identifier shortcut. Earlier global renaming and CSV row-order shuffling were insufficient because the oracle remained globally mapped to one identifier and canonical sorting could restore a fixed order. We therefore perform a full online DeepSeek 264×5 rerun on a per-event identifier-permutation variant while keeping the complete method frozen.

With seed 20260902, only candidate identifiers are reassigned within each event; display values, operations, and semantics remain unchanged. The event-level oracle-ID distribution becomes

$$(n_{001}, n_{002}, n_{003}) = (71, 88, 105).$$

An always-CAND_002 strategy would therefore achieve only $88/264 = 33.33\%$.

The full online result is

$$\text{Closure} = \frac{1315}{1320} = 99.62\%, \quad \text{Strict} = \frac{262}{264} = 99.24\%.$$

Oracle-correct and closure are identical at 1315/1320 (99.62%). We observe $N_{\text{wrong}} = 0$ and $N_{\text{abstain}} = 5$. The identifiers selected in the 1315 successful runs are

$$(\hat{n}_{001}, \hat{n}_{002}, \hat{n}_{003}) = (355, 436, 524),$$

showing that selections spread across all three identifiers in accordance with candidate content rather than concentrating on CAND_002. The oracle-ID distribution of the five abstentions is (0, 4, 1). For H5_E003, runs 2–5 terminate with IR_FAIL_CLOSED;

Table 14: Backend robustness on the same v5 benchmark

Backend	Closure	Strict events	Temporal	Rule-Exception	Cross-Sentence
DeepSeek	99.92%	262/264 (99.24%)	88/88	87/88	88/88
Qwen3.5:9b	99.24%	262/264 (99.24%)	88/88	88/88	86/88

for H5_E017, run 2 terminates with IR_RANK_TIE_ABSTAIN. Neither M14 nor M16 has an API error or timeout in this experiment.

At the event level, 262 events are strict successes in both original and permutation settings, H5_E003 succeeds only in the original setting, H5_E017 fails strict success in both, and no event succeeds only under permutation. A 10,000-replicate paired event-clustered bootstrap gives an Original-Permutation closure difference of +0.30 pp (95% CI: 0.00–0.91 pp) and a strict-event difference of +0.38 pp (95% CI: 0.00–1.14 pp). Thus the full online permutation produces only a small performance decrease, and a literal CAND_002/fixed-identifier shortcut cannot explain the observed 99%+ performance.

6.5 RQ6: Does ECR-Repair Depend on Evidence and Structured Reasoning?

Evidence dependence analysis (Exp8). To test whether ECR-Repair relies on normative evidence rather than superficial candidate patterns, we compare the frozen pipeline under five evidence conditions. With original evidence, ECR achieves 1315/1320 successful closures (99.62%). Removing evidence reduces closure to 2.20%, masking source information yields 3.18%, shuffling evidence yields 31.89%, and replacing evidence with irrelevant same-source material yields 5.23%. The event-clustered bootstrap differences between normal evidence and the four counterfactual conditions are all positive, confirming that the method is evidence-dependent rather than evidence-agnostic.

Strong LLM baselines (Exp9). We compare ECR against three stronger alternatives: direct LLM selection with evidence, M16-only verification, and a compute-matched multi-call baseline. ECR obtains 99.62% closure accuracy, outperforming direct LLM (92.73%), compute-matched reasoning (89.85%), and M16-only verification (75.08%). Event-clustered bootstrap confirms statistically significant improvements over all baselines. These results indicate that additional LLM calls alone cannot reproduce the benefit of explicit semantic decomposition, symbolic constraints, and selective verification.

6.6 RQ5: What Are the Annotation and Provenance Boundaries of the Benchmark?

Candidate shortcut audit and benchmark reliability. A reliable benchmark must avoid both identifier artifacts and ambiguous human judgments. We therefore perform two complementary audits. First, the candidate-identifier permutation experiment (Exp10) tests whether the fixed oracle identifier in the original v5 construction can explain the observed performance. Under per-event permutation, oracle identifiers are redistributed as (71, 88, 105) over CAND_001, CAND_002, and CAND_003. The frozen ECR pipeline obtains 1315/1320 successful closures (99.62%) and 262/264 strict events (99.24%), while a literal always-CAND_002 strategy achieves

only 33.33%. Candidate-only prediction without evidence obtains 0% closure. These results indicate that identifier or candidate-text shortcuts cannot explain the final performance.

Independent human annotation study. To validate benchmark labels, we conduct an independent human annotation study on 90 sampled events (30 TEMPORAL_VERSION, 30 GENERAL_RULE_EXCEPTION, and 30 CROSS_SENTENCE_SCOPE). Two human annotators independently label semantic type, evidence sufficiency, and oracle candidate without accessing model outputs or private evaluation information. The agreement is 100% for all evaluated fields, with Cohen’s $\kappa = 1.0$ for semantic type, evidence sufficiency, and oracle candidate. This result supports the reliability of the frozen benchmark annotations.

Historical provenance evaluation. The original v5 benchmark primarily evaluates evidence-relative semantic repair. To examine whether the approach also applies to temporally evolving specifications, we construct a strict historical provenance subset using same-family older RFC evidence and manual temporal validation. The subset contains 60 TEMPORAL_VERSION events and 300 formal attempts. ECR achieves 300/300 closure, 60/60 strict-event success, zero abstentions, and zero wrong selections. We report this result as a historical provenance subset evaluation rather than a complete provenance-chain benchmark because explicit replacement metadata is unavailable for many RFC cases.

Semantic IR fidelity and statistical reliability. We further audit whether the intermediate neural-to-symbolic representation captures the intended semantics. Human-gold comparison shows that Temporal Anchor Recovery substantially improves task-relevant temporal normalization: effective-anchor accuracy increases from 8.0% to 100.0%, and complete temporal frame accuracy increases from 2.3% to 38.6%. However, relation-type reconstruction remains incomplete, indicating that the module should be interpreted as task-oriented semantic normalization rather than a general temporal information extractor.

Finally, statistical audits confirm that the main result is stable under alternative uncertainty analyses. Source-cluster bootstrap over the 264-event benchmark gives a closure confidence interval of [99.02%, 100%]. Leave-one-source-out evaluation ranges from 99.62% to 99.85%. No wrong selections are observed in 1320 attempts; the Clopper-Pearson 95% upper bound of the conditional wrong-selection rate is 0.23%.

6.7 Failure Modes and Test-Slice Analysis

Domain and source-family slices. Ten of the 11 domains achieve 100% closure. The only non-perfect domain is dns_security, at 119/120 (99.17%). Twelve of the 13 RFC source families represented in the final benchmark achieve 100%; the only non-perfect source family is IETF RFC 4035, also at 119/120. These are frozen-method

Table 15: Candidate-identifier sensitivity and shortcut audit

Setting	Oracle-ID distribution	Closure	Strict event	Interpretation
Original	0/264/0	99.62%	262/264 (99.24%)	constant identifier; 0 wrong, 1 abstain
Global rename	0/264/0 equivalent	99.47%	261/264	oracle identity remains constant; 0 wrong, 7 abstain
Per-event permutation (online)	71/88/105	99.62%	262/264	always-CAND_002=33.33%; 0 wrong, 5 abstain

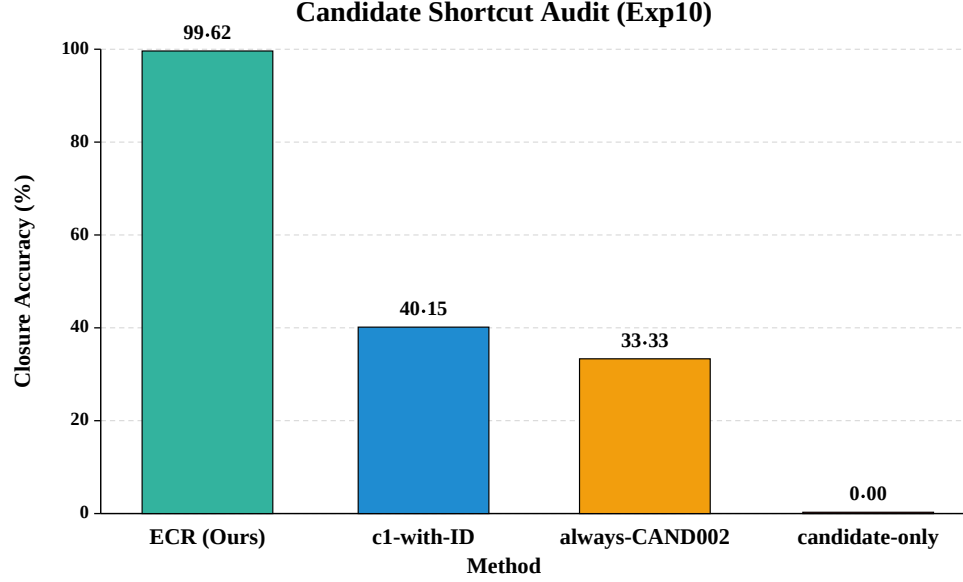
**Figure X.** Candidate shortcut audit (Exp10). ECR maintains high performance, whereas ID-leakage and shortcut baselines collapse, confirming that the benchmark cannot be solved by identifier bias.

Figure 6: Candidate shortcut audit. Identifier-based strategies achieve substantially lower performance than ECR-Repair, indicating that the main results cannot be explained by fixed candidate identifiers.

Table 16: Evidence-dependence analysis under counterfactual evidence conditions

Condition	Closure	Abstain
Normal	99.62%	5/1320
No evidence	2.20%	1271/1320
Masked source	3.18%	1258/1320
Shuffled evidence	31.89%	854/1320
Irrelevant same-source	5.23%	1221/1320

test-slice stability results, not leave-one-domain-out training experiments or strict cross-domain transfer/generalization evaluations.

Failure case and failure chain. The primary v5 experiment has one failed run, H5_E017 run 5, a General Rule–Exception case in which the Fallback Candidate Entailment Verifier cannot identify exactly one supported candidate and therefore abstains. No wrong-selected patch is produced, and no correctly selected candidate subsequently fails OWL closure.

6.8 Auditable Decision-Chain Case Analysis

Aggregate results establish overall effectiveness, but audibility also requires tracing why an individual decision succeeds or abstains. We therefore use the running successful example and two observed abstention trajectories to illustrate the audit interface without introducing additional performance samples.

Illustrative successful chain. Return to the $X \rightarrow Y$ running example from Section 3. Evidence focusing first retains the spans that

Table 17: Strong LLM baseline comparison

Method	Closure	Strict events
ECR-Repair	1315/1320 (99.62%)	262/264
Direct LLM	1224/1320 (92.73%)	228/264
Compute-matched	1186/1320 (89.85%)	204/264
M16-only	991/1320 (75.08%)	189/264

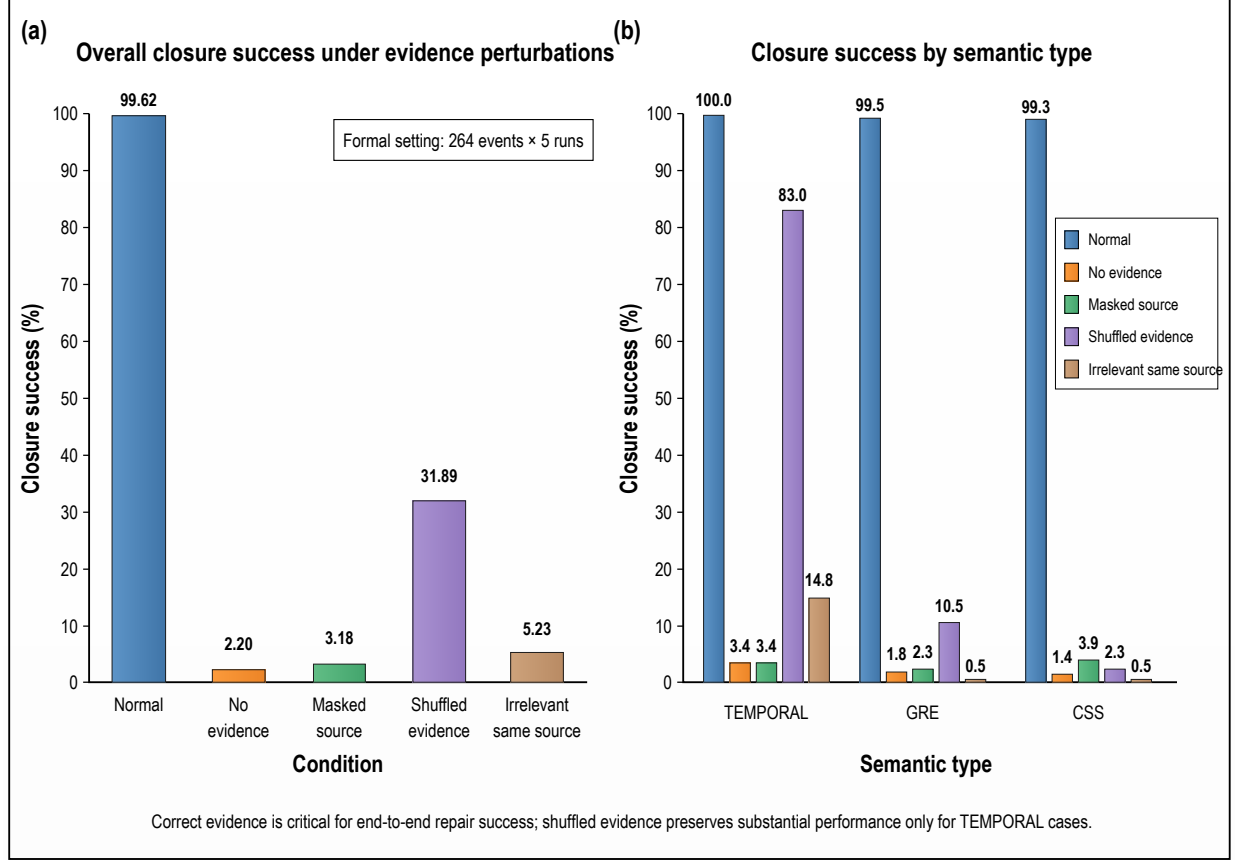
Experiment 8: Evidence Dependence of ECR-Repair

Figure 7: Evidence-dependence analysis under five evidence conditions. Removing or perturbing normative evidence substantially reduces repair success, demonstrating that ECR-REPAIR relies on evidence-grounded semantic reasoning rather than candidate artifacts.

Table 18: Strict historical provenance subset evaluation.

Setting	Events	Attempts	Result
Strict historical provenance subset	60	300	300/300 closure (100%)
Strict event success	60	–	60/60 (100%)
Wrong selection	–	300	0
Selective abstention	–	300	0

state “ Y supersedes X ” and identify Y as the current specification.

Semantic IR records X as superseded and Y as current. Candidate

Table 19: Marginal failure-state counts: Base Semantic IR Pipeline vs. ECR-REPAIR

Failure/success state	Base Semantic IR Pipeline	ECR-REPAIR
IR invalid / incomplete	219	0
Rank/tie abstain	67	1
Wrong select	12	0
OWL closure fail after correct select	0	0
Success	1022	1319

Failure-State Marginals: Base Semantic IR vs. ECR-Repair

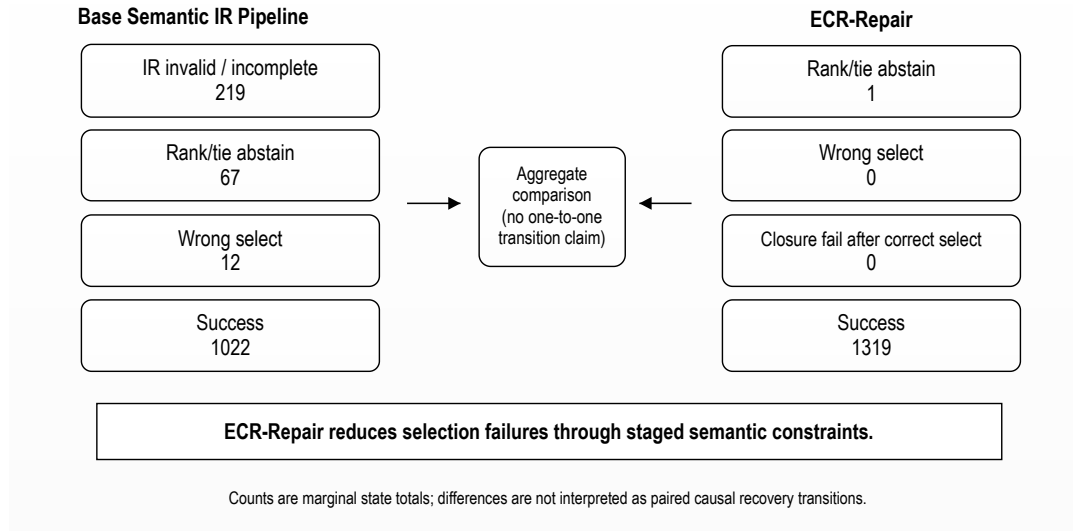


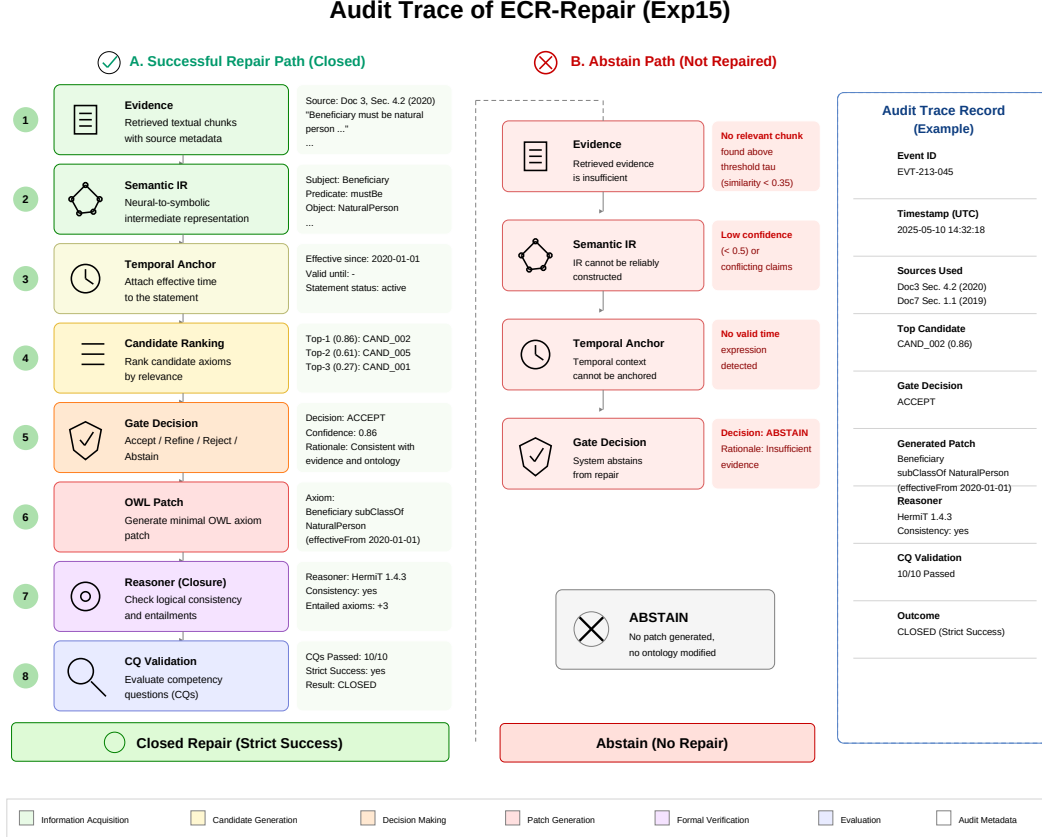
Figure 8: Marginal failure-state compression from the Base Semantic IR Pipeline to ECR-REPAIR. Counts summarize aggregate state totals and are not interpreted as one-to-one paired causal recovery transitions.

scoring can then explain why “update to Y ” is preferred over “retain X ” or the neighboring distractor Z through token/exact matches, specification identifiers, type-specific semantic consistency, and directional-conflict penalties. Once a unique candidate passes the frozen gate, formal closure checks only the declared deletion/addition graph delta, reasoner consistency, and frozen regression conditions. The chain illustrates the audit granularity of ECR-REPAIR: natural-language interpretation, candidate evidence, and formal execution are retained in different responsibility layers.

Observed abstention trajectories. In the main v5 run, H5_E017 is a General Rule–Exception event for which Semantic IR is available but candidate ranking ties; fallback candidate entailment verification still cannot identify a unique supported candidate, so the system returns ABSTAIN rather than executing a formally possible but evidentially underdetermined patch. In the online candidate-ID permutation experiment, four failures of H5_E003 stop earlier at INCOMPLETE_IR/IR_FAIL_CLOSED. Neither failure mode is caused by an API error or timeout. The abstentions are therefore diagnosable semantic/decision outcomes rather than infrastructure failures. Table 20 summarizes the representative trajectories.

Table 20: Representative auditable trajectories: one successful example and observed abstentions

Case	Key state	Decision path	Audit interpretation
Running $X \rightarrow Y$ example H5_E017	current/superseded roles IR OK, rank tie	IR $\rightarrow$ score components $\rightarrow$ unique gate $\rightarrow$ closure fallback finds no unique supported candidate $\rightarrow$ ABSTAIN	successful decision can be decomposed into evidence, candidate features, and formal execution reject a patch when candidate evidence is insufficient
H5_E003 (permutation test)	incomplete IR	IR_FAIL_CLOSED $\rightarrow$ ABSTAIN	stop before candidate execution when semantic representation is insufficient

**Figure 9: End-to-end audit trace of ECR-Repair. Each decision is connected from evidence interpretation through Semantic IR, candidate evaluation, formal closure, and CQ validation.**

7 Discussion

The central issue in this task is not simply whether an LLM can produce structurally valid output, but whether the overall system exposes a stable and diagnosable neural-to-symbolic semantic interface. The design question in ECR-REPAIR can be summarized as *who is responsible for what, when, and on what evidence*. Neural components interpret normative evidence and resolve residual semantic ambiguity. Symbolic components constrain those interpretations into Typed Semantic IR, deterministic temporal relations, and candidate-admissibility conditions. Formal closure is responsible only for determining whether the OWL/RDF edit can be executed

safely. Candidate-content isolation prevents candidate semantics from contaminating the earlier neural interpretation. The final repair is therefore not the direct product of one LLM generation but the output of a layered decision process with explicit responsibility boundaries.

The additional validation clarifies the scope of these responsibilities. First, the human temporal-gold audit shows that Temporal Anchor Recovery is most valuable for task-relevant anchor roles such as effective and current, rather than full relation-graph reconstruction: effective-anchor accuracy rises from 8% to 100%, while relation typing decreases. This suggests that local repair can

rely on **sufficient structured semantics** rather than exhaustive reconstruction of every temporal fact. Second, the formal-closure stress test shows that closure reliably blocks undeclared graph edits, logical inconsistency, and frozen regression violations, while failing to recognize patches that are semantically wrong yet formally self-consistent. Semantic correctness must therefore remain the responsibility of evidence-grounded candidate selection; formal closure cannot substitute for a semantic oracle.

The candidate-pool and backend experiments also narrow the interpretation that the 99.92% result depends on a single narrow configuration. Increasing the candidate pool from three to ten reduces strict event success only to 260/264, with Temporal-Version remaining perfect. Replacing the cloud backend with local Qwen3.5:9b on the same v5 benchmark yields 262/264 strict event success, one event below DeepSeek. These results support robustness to finite candidate-pool enlargement and limited backend portability, but not open-ended generation, arbitrarily large candidate spaces, or model independence.

From a selective-prediction perspective, the abstention mechanism instantiates the principle “prefer no repair over an unsupported repair” at one frozen operating point. On the main v5 benchmark, Coverage is 99.92% and $P_{\text{sel}} = 100\%$, corresponding to zero observed conditional semantic risk among selected cases. Because we do not sweep thresholds to produce a complete risk-coverage curve, the evidence supports an operational selective-repair mechanism rather than conclusions about an optimal risk function or calibrated selector.

The candidate-identifier audit exposes and directly addresses an important benchmark regularity: the v5 construction template assigns every oracle to CAND_002. Global renaming and row-order shuffling do not fully remove this risk because the oracle identity remains globally constant. The stronger per-event permutation redistributes oracle IDs to 71/88/105 and, under a frozen M13–M16 pipeline, a full online DeepSeek 264×5 rerun still obtains 99.62% closure, 262/264 strict event success, and zero wrong selections; a literal always-CAND_002 strategy would achieve only 33.33%. Paired event-clustered differences are small as well (closure +0.30 pp, 95% CI 0.00–0.91; strict +0.38 pp, 95% CI 0.00–1.14, Original minus Permutation). This rules out the literal fixed-identifier shortcut as an explanation for the observed 99%+ performance. It does not prove that the benchmark contains no other construction regularities.

Finally, the strict provenance audit result of $N = 0$ is an important boundary rather than a disposable null result. The current v5 benchmark is built from real RFC evidence and supports evidence-relative repair decisions, but under the frozen evidence cache it cannot reconstruct a provenance-complete $t_0 \rightarrow t_1$ historical drift chain for each event. We retain “semantic drift” as the problem background and evidence-relative operational concept, but the 99.92% figure must not be interpreted as the success rate of historical drift discovery or provenance reconstruction.

8 Threats to Validity

Source-domain limitation. The final benchmark is dominated by public RFC/IETF-style technical specifications. Although it covers 11 domains and 13 source families, the results do not directly

imply equal effectiveness for legal instruments, contracts, clinical guidelines, or other normative genres.

Data construction and identifier regularity. v5 uses real RFC documents and evidence windows, but repair events, subject/predicate metadata, and candidate sets are still created under an experimental protocol. The frozen template also assigns all oracles to CAND_002. A full online 264×5 per-event identifier-permutation run explicitly audits this regularity: after redistributing the oracle IDs to 71/88/105, closure remains 99.62%, strict event success remains 262/264, and wrong selection remains zero. A literal fixed-identifier strategy therefore cannot explain the result, but the benchmark-construction regularity itself must remain transparently disclosed.

Annotation-reliability boundary. A 90-event independent human annotation study is performed with two human annotators. Agreement is 100% for semantic type, evidence sufficiency, and oracle candidate, with Cohen’s $\kappa = 1.0$. The study validates the sampled benchmark labels, but it does not imply that all possible domain interpretations outside the sampled semantic categories are unambiguous.

Semantic-drift provenance. The stricter provenance audit requiring explicit lineage, t_0 support for the old assertion, and t_1 rejection of the old assertion together with support for the replacement yields $N = 0$. The main experiment therefore supports evidence-relative semantic-drift repair, not provenance-complete historical drift reconstruction. This limitation is partly caused by incompleteness of the current public RFC cache.

Conditional-task boundary. v5 assumes that a potential event has already been localized, a relevant evidence window has been provided, and the correct repair occurs in the finite candidate set. We do not evaluate open-corpus retrieval recall, candidate-generation recall, or drift-discovery precision/recall/F1. The $k = 5/10$ hard-negative variants show stability for larger finite pools but do not justify conclusions about open-ended generation or arbitrary candidate spaces.

Formal-closure boundary. The closure stress test rejects all structurally invalid patch classes involving extra graph edits, reasoner inconsistency, or repair-check failure, but all 264 semantically wrong yet formally self-consistent patches still pass closure. Formal closure provides structural/logical/regression safety and cannot replace evidence-grounded semantic selection.

CQ/repair-check independence. The frozen regression layer is dominated by event-specific triple-level repair checks constructed under the same event protocol. It is not an independent external semantic gold set. A stronger evaluation should include a richer CQ/SPARQL suite authored independently of repair-event construction.

Scope of Temporal Recovery. On human gold, M15 substantially improves effective-anchor and role-frame metrics but decreases relation-type accuracy. We therefore position it as task-oriented temporal-anchor normalization rather than a general, complete temporal relation-extraction system.

Ontology scale and reasoning scalability. The unit of evaluation is an event-level local patch rather than an ontology-size, TBox-depth, or large-ABox scalability study. The results therefore do not establish global reasoning scalability for very large ontologies.

Candidate-content-isolation boundary. The General Rule–Exception and Cross-Sentence Scope paths allow one `ranking_failed=true` Boolean to trigger a recovery retry. The candidate set can therefore influence whether a retry occurs, although concrete candidate content is never fed back into semantic recovery. We explicitly call this mechanism candidate-content isolation rather than absolute candidate independence.

Statistical independence. The five runs of one event are not independent samples. Attempt-level percentages are therefore descriptive, primary uncertainty and stage differences use event-clustered bootstrap, and strict event success uses the event as the independent unit with a Wilson interval. Robustness percentages without clustered confidence intervals are treated descriptively.

Model backends and timeout behavior. DeepSeek is a mutable cloud-service model identifier, so future backend updates may affect exact reproducibility. The Qwen3.5:9b v5 run has 15/1320 Base IR timeouts, all counted as failures without selective reruns. Similar overall performance supports portability but not model independence.

Interpretation of the candidate-ID permutation test. The full online permutation test rules out one specific explanation of the 99%+ performance: always selecting literal CAND_002 or another fixed identifier. It does not prove that no other shortcut exists, nor does it alter the fact that candidate construction is protocol-designed. We therefore interpret the experiment as counterfactual robustness evidence against literal candidate-identifier bias.

Compute and baseline fairness. The simple candidate-selection baselines are effectiveness controls, not compute-matched systems. ECR-REPAIR may perform recovery or additional verifier calls, so the current experiments do not establish token- or latency-efficiency advantages.

Implementation metadata. The manuscript fixes the logical materialization–reasoner–graph-delta–repair-check interface but does not list software versions for unverified reasoner/RDF-library/harness components. A formal artifact release should provide these environment and dependency details; until then, we do not claim cross-reasoner or cross-library reproduction equivalence.

Final blind-evaluation boundary. v1/v3/v4 are diagnostic, calibration, or supplementary validation data. v5 is the primary blind benchmark rebuilt and frozen after the complete method freeze. The paper-final validation performs additional audits and robustness tests on the frozen method and does not retune M13–M16, ranking thresholds, or prompts in response to those results.

9 Conclusion

We presented ECR-REPAIR, an evidence-constrained modular neuro-symbolic ontology repair framework for evolving normative documents. Semantic IR acts as the neural-to-symbolic semantic interface: LLMs perform candidate-content-isolated evidence interpretation and on-demand candidate entailment verification; deterministic recovery and constraint-aware ranking implement symbolic semantic decision making; and OWL/RDF materialization, reasoning, graph-delta validation, and frozen repair/CQ regression checks implement formal closure. Under the primary conditional v5 setting—the potential event is localized, relevant evidence is

provided, and the correct repair appears in a finite candidate set—264 RFC-based repair events and 1320 runs yield 99.92% closure accuracy, 262/264 (99.24%) strict event success, and zero wrong-selected patches.

The final validation shifts the interpretation from “high accuracy” to “high accuracy with auditable boundaries.” Human temporal gold supports substantial improvement in effective-anchor recovery but not characterization of Temporal Anchor Recovery as a complete temporal relation extractor. Formal-closure stress testing demonstrates detection of structural, logical, and regression violations but not semantic errors that remain formally self-consistent. Strict event success remains 260/264 at $k = 10$, and a local Qwen3.5:9b backend reaches 262/264. Most importantly, a full online DeepSeek run on a per-event candidate-ID permutation breaks the fixed CAND_002 regularity yet still obtains 1315/1320 (99.62%) closure, 262/264 (99.24%) strict success, and zero wrong selections, compared with only 33.33% for an always-CAND_002 shortcut. At the same time, the strict historical-provenance audit admits $N = 0$ events, so our claim remains evidence-relative semantic-drift repair rather than provenance-complete historical drift reconstruction.

More broadly, for normative knowledge engineering settings in which decisions must be auditable and attributable, the design question for a neuro-symbolic system should not be reduced to “which layer is more intelligent?” but should instead ask *which layer is responsible for which decision, at what stage, and on what evidence*. Reliable ontology repair cannot rely solely on a stronger language model, nor can it treat formal consistency as equivalent to semantic correctness. A more defensible path is to establish explicit responsibility boundaries through Semantic IR, candidate-content isolation, selective abstention, symbolic constraints, and formal safety checks. The present conclusions do not cover drift discovery, open-corpus retrieval, open-ended candidate generation, arbitrarily large candidate spaces, or large-scale ontology reasoning.

Generative AI Use Statement

OpenAI ChatGPT was used to assist with language editing, structural organization, academic terminology, LaTeX formatting, and presentation refinement. The authors reviewed and edited the research questions, experimental design, data, analysis, conclusions, and final manuscript, and remain responsible for the accuracy and integrity of all content.

References

- [1] Shqiponja Ahmetaj, Robert David, Axel Polleres, and Mantas Šimkus. 2022. Repairing SHACL Constraint Violations Using Answer Set Programming. In *The Semantic Web – ISWC 2022 (Lecture Notes in Computer Science, Vol. 13489)*. Springer, 375–391. doi:10.1007/978-3-031-19433-7_22
- [2] Shqiponja Ahmetaj, Robert David, Axel Polleres, and Mantas Šimkus. 2025. A Logic Programming Approach to Repairing SHACL Constraint Violations. *Transactions on Graph Data and Knowledge* 3, 3 (2025), 1:1–1:36. doi:10.4230/TGDK.3.3.1
- [3] Viviane Torres da Silva, Jéssica Soares dos Santos, Raphael Thiago, Elton Soares, and Leonardo Guerreiro Azevedo. 2022. OWL Ontology Evolution: Understanding and Unifying the Complex Changes. *The Knowledge Engineering Review* 37 (2022), e10. doi:10.1017/S0269888922000066
- [4] Wenqi Fan, Yujuan Ding, Liangbo Ning, Shijie Wang, Hengyun Li, Dawei Yin, Tat-Seng Chua, and Qing Li. 2024. A Survey on RAG Meeting LLMs: Towards Retrieval-Augmented Large Language Models. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. ACM, 6491–6501. doi:10.1145/3637528.3671470

- [5] Nadeen Fathallah, Arunav Das, Stefano De Giorgis, Andrea Poltronieri, Peter Haase, Liubov Kovriguina, Albert Meroño-Peñuela, Elena Simperl, Steffen Staab, and Alsayed Algergawy. 2026. Extended NeOn-GPT: Advancing LLM-Powered Ontology Learning Through Ontology Reuse and Automated Verification. *Semantic Web* 17, 4 (2026), 1–47. doi:10.1177/22104968261453138
- [6] Zhengyao Gu and Mark Hopkins. 2023. On the Evaluation of Neural Selective Prediction Methods for Natural Language Processing. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 7888–7899. doi:10.18653/v1/2023.acl-long.437
- [7] Keerat Guliani, Deepkamal Gill, David Landsman, Nima Eshraghi, Krishna Kumar, and Lovedeep Gondara. 2026. De Jure: Iterative LLM Self-Refinement for Structured Extraction of Regulatory Rules. arXiv:2604.02276 arXiv preprint.
- [8] Qiu Ji, Guilin Qi, Yinkai Yang, Weizhuo Li, Siying Huang, and Yang Sheng. 2022. An Embedding-Based Approach to Repairing OWL Ontologies. *Applied Sciences* 12, 24 (2022), 12655. doi:10.3390/app122412655
- [9] Jiayi Li, Daniel Garijo, and María Poveda-Villalón. 2026. Large Language Models for Ontology Engineering: A Systematic Literature Review. *Semantic Web* 17, 4 (2026), 1–45. doi:10.1177/22104968261465514
- [10] Anna Sofia Lippolis, Mohammad Javad Saeedizade, Robin Keskisärkkä, Sara Zuppiroli, Miguel Ceriani, Aldo Gangemi, Eva Blomqvist, and Andrea Giovanni Nuzzolese. 2025. Ontology Generation Using Large Language Models. In *The Semantic Web – ESWC 2025 (Lecture Notes in Computer Science, Vol. 15718)*. Springer, 321–341. doi:10.1007/978-3-031-94575-5_18
- [11] Marcin Pietranik and Adrianna Kozierkiewicz. 2023. Methods of Managing the Evolution of Ontologies and Their Alignments. *Applied Intelligence* 53 (2023), 20382–20401. doi:10.1007/s10489-023-04545-0
- [12] Damith Premasiri, Tharindu Ranasinghe, Ruslan Mitkov, Mo El-Haj, and Ingo Frommholz. 2025. Survey on Legal Information Extraction: Current Status and Open Challenges. *Knowledge and Information Systems* 67 (2025), 11287–11358. doi:10.1007/s10115-025-02600-5
- [13] Anmol Singhal and Travis D. Breaux. 2025. Legal Requirements Translation from Law. In *2025 IEEE 33rd International Requirements Engineering Conference (RE)*. IEEE, 205–217. doi:10.1109/RE63999.2025.00028
- [14] Jiashuo Sun, Pengcheng Jiang, Saizhuo Wang, Jiajun Fan, Heng Wang, Siru Ouyang, Ming Zhong, Yizhu Jiao, Chengsong Huang, Xueqiang Xu, Pengrui Han, Peiran Li, Jiaxin Huang, Ge Liu, Heng Ji, and Jiawei Han. 2026. Rethinking the Reranker: Boundary-Aware Evidence Selection for Robust Retrieval-Augmented Generation. In *Proceedings of the 43rd International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 306)*.
- [15] Sinh Trong Vu, Minh Le Nguyen, and Ken Satoh. 2022. Abstract Meaning Representation for Legal Documents: An Empirical Research on a Human-Annotated Dataset. *Artificial Intelligence and Law* 30, 2 (2022), 221–243. doi:10.1007/s10506-021-09292-6
- [16] W3C OWL Working Group. 2012. OWL 2 Web Ontology Language Document Overview (Second Edition). W3C Recommendation. 11 December 2012.
- [17] World Wide Web Consortium. 2017. Shapes Constraint Language (SHACL). W3C Recommendation. 20 July 2017.
- [18] Nan Wu, Xiacong Lai, Mei Chen, and Ying Pan. 2024. Ontology Matching and Repair Based on Semantic Association and Probabilistic Logic. *IEICE Transactions on Information and Systems* E107-D, 11 (2024), 1433–1443. doi:10.1587/transinf.2024EDP7028